

EL GRAN ESCAPE

Guión del video de presentación - 7 personas - 2:30 min cada uno

Proyecto final: juego multijugador de persecución en un laberinto procedural. 3 escapistas vs 2 cazadores. Python 3 + Pygame (pygame-ce) + PostgreSQL 17 (psycopg 3). El multijugador se sincroniza por completo a través de la base de datos, sin servidor de juego.

Quién	Tema	Tiempo	Qué muestra
Persona 1	Estructura principal + demo	2:30	main.py, el juego y README.md
Persona 2	Laberinto procedural	2:30	juego/laberinto.py + demo ASCII en consola
Persona 3	Motor: física y radar	2:30	juego/entidad.py, camara.py, render.py
Persona 4	Reglas y bots	2:30	juego/partida.py, bots.py, tests_fase2.py
Persona 5	PostgreSQL y ranking	2:30	db/esquema.sql, conexion.py, repositorio.py
Persona 6	Multijugador y lobby	2:30	db/red.py, juego/partida_multi.py + demo
Persona 7	Arbitraje y pruebas	2:30	db/repositorio.py, tests_fase4.py

Cómo usar este guión: cada sección tiene tres partes. **EN PANTALLA** es lo que tiene que verse mientras hablás. **GUIÓN** es el texto sugerido, con marcas [MOSTRÁ: ...] para cambiar el contenido visual. **SI EL PROFESOR PREGUNTA** es solo de preparación.

Asignen los nombres acá: P1 _____, P2 _____, P3 _____, P4 _____, P5 _____, P6 _____, P7 _____.

PERSONA 1 - Estructura principal y demo del juego (2:30)

Abrir el video: mostrar la estructura del código en main.py y una demo rápida del juego.

EN PANTALLA (en este orden)

- main.py: la definición de las constantes de estado (línea 28) y el loop principal (while corriendo).
- Tener el juego ya abierto (python main.py).
- Pantalla de nombre, menú, partida rápida de escapista, pantalla fin y ranking (tecla T).
- README.md mostrando el plan de fases.

GUIÓN (lo que decís)

Hola, somos el grupo [nombres] y presentamos El Gran Escape. Un juego multijugador asimétrico en Python, Pygame y PostgreSQL. Tres escapistas buscan salidas y dos cazadores los persiguen.

[MOSTRÁ: main.py, línea 28 y el while corriendo.] A nivel código, el flujo funciona con una máquina de estados. Dependiendo de si estamos en MENU, LOBBY o JUGANDO, el loop principal procesa los eventos del teclado de manera distinta y le pide al módulo render que dibuje la pantalla correspondiente.

[MOSTRÁ: el juego.] Desde el menú podés jugar solo o ir al multijugador por base de datos, que luego explicaremos. Juguemos una partida rápida.

[MOSTRÁ: jugá 30 segs de escapista.] El laberinto se genera solo. La cámara sigue al jugador. Abajo está el radar: los cazadores son más rápidos, pero con el radar podemos anticiparlos.

[MOSTRÁ: dejate atrapar o escapá, pantalla de fin, luego T.] Al final el resultado se guarda y vemos el ranking global.

[MOSTRÁ: README.md.] Desarrollamos esto en cuatro fases, y así nos repartimos la explicación. [Persona 2] arranca con el laberinto.

SI EL PROFESOR PREGUNTA (no va en el video)

P: ¿Por qué un laberinto tan grande no es injugable?

R: La cámara centrada y el minimapa evitan que sea frustrante, lo fuimos iterando.

PERSONA 2 - El laberinto procedural (2:30)

Explicar generación, braiding y el concepto de la seed.

EN PANTALLA (en este orden)

- juego/laberinto.py: `_excavar()`, `_abrir_atajos()` y `__init__`.
- Consola: `python -m juego.laberinto`

GUIÓN (lo que decís)

[MOSTRÁ: juego/laberinto.py, _excavar().] Usamos Recursive Backtracking para generar el mapa. Como la recursión nativa en Python rompería por límite de profundidad en mapas tan grandes, lo programamos iterativo usando una pila explícita.

[MOSTRÁ: _abrir_atajos().] Un laberinto 'perfecto' tiene una sola ruta entre dos puntos. Para un juego de persecución eso es fatal porque te bloquean fácil. Aplicamos 'braiding': abrimos un 20% de los callejones sin salida para crear múltiples rutas.

[MOSTRÁ: __init__.] Todo el azar sale de una semilla. Misma seed, mismo laberinto. En el multijugador no mandamos por red el mapa, solo la seed y cada cliente lo recrea.

[MOSTRÁ: python -m juego.laberinto.] Este módulo incluye una demo ASCII que verifica con un BFS que las 3 salidas sean siempre alcanzables. Sigue [Persona 3].

SI EL PROFESOR PREGUNTA (no va en el video)

P: ¿Qué pasa si la seed es la misma?

R: El nivel es idéntico. Se puede probar lanzando el juego con `--seed 1234`.

PERSONA 3 - Motor: física, cámara y radar (2:30)

Explicar colisiones por ejes, recorte de cámara y el radar.

EN PANTALLA (en este orden)

- juego/entidad.py: mover().
- juego/camara.py: a_pantalla().
- juego/render.py: dibujar_laberinto() y dibujar_minimapa().

GUIÓN (lo que decís)

[MOSTRÁ: juego/entidad.py.] Tanto humanos como bots instancian la misma clase Entidad y física. Ningún bot puede hacer trampa ni traspasar paredes.

[MOSTRÁ: mover().] Resolvemos colisiones eje por eje (X primero, luego Y). Así al ir en diagonal contra un muro, te deslizás suavemente. Y normalizamos el vector para no movernos más rápido en las diagonales.

[MOSTRÁ: camara.py y render.py.] El laberinto tiene más de 10.000 tiles. La cámara sigue al jugador y dibujamos sólo lo visible. Así garantizamos 60 FPS estables.

[MOSTRÁ: dibujar_minimapa().] Para el balance, el cazador es un 9% más rápido, pero el escapista tiene ventaja de radar: las salidas siempre se ven, y los enemigos sólo si están cerca. Le toca a [Persona 4].

SI EL PROFESOR PREGUNTA (no va en el video)

P: ¿Por qué no usar sprites en vez de círculos?

R: Priorizamos las lógicas y mecánicas. Todo está desacoplado en render.py para cambiarse fácil.

PERSONA 4 - Reglas del juego e IA de los bots (2:30)

Explicar bucle lógico separado de Pygame y la decisión de IA de los bots.

EN PANTALLA (en este orden)

- juego/partida.py: actualizar().
- juego/bots.py: CerebroCazador y CerebroEscapista.
- Consola: python tests_fase2.py.

GUIÓN (lo que decís)

[MOSTRÁ: juego/partida.py.] Las reglas están 100% separadas de Pygame. Cada ciclo actualiza posiciones y chequea capturas o escapes.

[MOSTRÁ: juego/bots.py.] Los cazadores patrullan al azar, y persiguen con BFS sólo si tienen línea de visión directa. El escapista fue un reto: al principio usaban el camino óptimo y ganaban siempre en segundos. Los empeoramos: ahora exploran y si una salida entra a su radar, recién ahí van a ella.

[MOSTRÁ: tests_fase2.py.] Como la lógica no depende de gráficos, hicimos este script que corre 5 partidas de bots contra bots en segundos, para comprobar el balanceo final. Paso a [Persona 5].

SI EL PROFESOR PREGUNTA (no va en el video)

P: ¿Por qué usan BFS y no A* para el camino?

R: En un grid sin pesos, BFS te da la ruta más corta y es súper eficiente. A* no era necesario.

PERSONA 5 - PostgreSQL: esquema, guardado y ranking (2:30)

Explicar tablas UNLOGGED, creación automática y tolerancia a fallos.

EN PANTALLA (en este orden)

- db/esquema.sql: posiciones UNLOGGED y vistas.
- db/conexion.py y repositorio.py.
- Juego: pantalla de ranking.

GUIÓN (lo que decís)

[MOSTRÁ: db/esquema.sql.] Nuestro modelo tiene 4 tablas. La tabla clave es posiciones, que es UNLOGGED. Se usa para el multijugador porque son datos efímeros de alta rotación. Al ser unlogged, no escriben en el diario de transacciones (WAL), haciendo las escrituras ultra rápidas.

[MOSTRÁ: db/conexion.py.] La base se autoconfigura al conectarse. Si hay error de red, lanza DBNoDisponible, el juego lo atrapa y sigue. La caída de la base no congela la partida.

[MOSTRÁ: db/repositorio.py y juego ranking.] Guardamos a los 5 participantes marcando quién es bot. Las vistas de SQL los filtran y el ranking usa MIN(tiempo) y GROUP BY para mostrar sólo el récord de cada persona y evitar monopolios. Sigue [Persona 6].

SI EL PROFESOR PREGUNTA (no va en el video)

P: ¿Qué es UNLOGGED?

R: Omite logs de transacción (WAL), ganando mucho rendimiento a costa de persistencia ante fallos críticos.

PERSONA 6 - Multijugador: lobby y sincronización (2:30)

Explicar que no hay server de juego sino PostgreSQL y el suavizado.

EN PANTALLA (en este orden)

- Dos instancias del juego conectadas en la misma sala (grabar demo).
- db/red.py y juego/partida_multi.py.

GUIÓN (lo que decís)

[MOSTRÁ: demo 2 ventanas online.] La mayor innovación es que el multiplayer NO tiene servidor de juego. Toda la red pasa por PostgreSQL. Acá vemos dos instancias jugando sincronizadas.

[MOSTRÁ: db/red.py.] Un hilo paralelo de red hace upsert de nuestra posición cada 100 ms y lee las enemigas. Así el loop del juego nunca frena esperando la red, manteniendo 60 FPS.

[MOSTRÁ: partida_multi.py.] A 10 Hz los rivales darían saltos. Lo solucionamos con 'suavizado exponencial', acercándolos progresivamente cada frame para lograr movimiento fluido. En cambio, nuestro propio jugador usa física local para moverse sin input lag.

Si faltan humanos en la sala, el cliente del 'host' los reemplaza simulando bots que envían sus posiciones a la DB, lo cual es transparente para el resto. Cierra [Persona 7].

SI EL PROFESOR PREGUNTA (no va en el video)

P: ¿Cuánta carga le pone a la DB?

R: Son unas 10 escrituras chicas por segundo por cliente. En una red local no genera impacto.

PERSONA 7 - Arbitraje atómico, pruebas y cierre (2:30)

Mostrar cómo Postgres arbitra las capturas de forma atómica y cerrar.

EN PANTALLA (en este orden)

- db/repositorio.py: intentar_captura().
- Consola: python tests_fase4.py.
- README.md: limitaciones.

GUIÓN (lo que decís)

Si un cazador dice 'te atrapé' y un escapista dice 'escapé', ¿quién gana ante el lag? No lo decide el cliente, lo decide la base de datos atómicamente.

[MOSTRÁ: intentar_captura().] Hacemos un UPDATE condicional. Verifica primero que las distancias almacenadas sean cortas. Postgres serializa estas transacciones. Si hay conflicto simultáneo, el primer UPDATE marca la captura y el segundo falla (rowcount 0).

[MOSTRÁ: python tests_fase4.py.] Corremos una simulación de clientes reales en hilos que colisionan al mismo tiempo. El test prueba que el arbitraje rechaza reclamos falsos o lejanos.

[MOSTRÁ: README.md.] Como conclusión, sabemos que por internet habría latencia de input, y si el host se desconecta, sus bots quedan quietos. Asumir limitaciones es parte de la ingeniería. Este fue nuestro proyecto El Gran Escape, ¡muchas gracias!

SI EL PROFESOR PREGUNTA (no va en el video)

P: ¿Qué mejorarían con más tiempo?

R: Migraríamos a LISTEN/NOTIFY en Postgres para bajar el polling y aplicaríamos un anti-cheat básico.

CONSEJOS DE GRABACIÓN Y EDICIÓN

Cómo grabar

- Graben cada bloque por separado (cada uno el suyo) y júntenlo en la edición; queda mucho mejor que una llamada grupal continua.
- Aumenten la fuente del editor (Ctrl + + un par de veces).
- Las demos del juego grábenlas ANTES y pongan la voz en post.
- Para la demo multijugador: dos consolas, python main.py, ventanas lado a lado.

Manejo del tiempo (Total ~17:30)

- Cada persona tiene exactamente 2:30 asignado para hablar y mostrar.
- No incluyan las preguntas 'SI EL PROFESOR PREGUNTA' en el video.

Comandos que se usan en el video

- python main.py
- python -m juego.laberinto
- python tests_fase2.py
- python tests_fase4.py

Regla de oro

- No lean textualmente: cuéntenlo natural. Las marcas [MOSTRÁ: ...] sí respétenlas porque indican qué debe verse.